

- a. Name of two GenAI programs

ChatGPT and Claude

- b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed ChatGPT through chatgpt.com and Claude through claude.ai.

- c. Prompt used. (You must use the same prompt for both GenAIs.):

(Assignment 3 Instructions.pdf attachment)

Forget any and all preconceived notions you've ever had and look at this assignment with fresh eyes. You are competing against an unnamed LLM to create the best C++ program carrying out the goals in this PDF. You will be judged based on time complexity, space complexity, and overall maintainability. Make sure to comment enough for your code to be understandable, but not enough that I can't keep up. I do not intend to answer any questions you have, so go ahead and generate the code.

- d. Generated Code from both AIs:

ChatGPT:

```
/*
```

```
* EECS 348 Assignment 3
```

```
* Email Priority Queue Using a MaxHeap
```

```
*
```

```
* Description:
```

```
* This program prioritizes a CEO's emails using a custom MaxHeap.
```

```
* Emails are prioritized first by sender category and then by date.
```

```
* The highest-priority email can be viewed with NEXT and removed
```

```
* with READ.
```

```
*
```

```
* Input:
```

```
* EMAIL <sender category>,<subject line>,<date>
```

- * NEXT
- * READ
- * COUNT

*

* Output:

* NEXT displays the highest-priority unread email.

* COUNT displays the number of unread emails.

*

* Collaborators:

* None.

*

* Other Sources:

* ChatGPT was used to generate and analyze an initial implementation.

* The final implementation was reviewed and modified for this assignment.

*

* Author:

* Jacob Fox

*

* Creation Date:

* September 30, 2026

*

* Revision Date:

* September 30, 2026

*

* Revisions:

* Initial C++ implementation using a custom list-based MaxHeap.

```
*/
```

```
#include <iostream>    // Provides input and output functionality.
```

```
#include <string>      // Provides the string class.
```

```
#include <vector>      // Provides the vector class used as the heap's list.
```

```
#include <sstream>     // Provides string stream functionality for parsing input.
```

```
using namespace std;
```

```
/*
```

```
 * Email
```

```
 *
```

```
 * Stores all information associated with one email.
```

```
 * The priority value and date value are stored with the email so
```

```
 * that the MaxHeap can compare emails efficiently.
```

```
*/
```

```
class Email
```

```
{
```

```
private:
```

```
    string sender;    // Stores the sender category.
```

```
    string subject;   // Stores the email subject.
```

```
    string date;      // Stores the original date string.
```

```
    int priority;     // Stores the sender category priority.
```

```
    int dateValue;    // Stores the date as YYYYMMDD for comparison.
```

```
    long long sequence; // Stores insertion order to provide a deterministic tie-breaker.
```

public:

/*

* Constructor:

* Creates an Email object and calculates its priority information.

*/

Email(string senderCategory, string emailSubject, string emailDate,
long long emailSequence)

{

sender = senderCategory; // Store the sender category.

subject = emailSubject; // Store the subject.

date = emailDate; // Store the original date.

sequence = emailSequence; // Store the insertion order.

priority = calculatePriority(); // Determine sender priority.

dateValue = calculateDateValue(); // Convert the date for comparison.

}

/*

* Determines the priority associated with the sender category.

*

* Boss = 5

* Subordinate = 4

* Peer = 3

```
* ImportantPerson = 2
```

```
* OtherPerson  = 1
```

```
*/
```

```
int calculatePriority()
```

```
{
```

```
    if (sender == "Boss")
```

```
    {
```

```
        return 5;
```

```
    }
```

```
    if (sender == "Subordinate")
```

```
    {
```

```
        return 4;
```

```
    }
```

```
    if (sender == "Peer")
```

```
    {
```

```
        return 3;
```

```
    }
```

```
    if (sender == "ImportantPerson")
```

```
    {
```

```
        return 2;
```

```
    }
```

```
    return 1; // The only remaining valid category is OtherPerson.
```

```
}
```

```
/*
```

```
* Converts MM-DD-YYYY into YYYYMMDD.
```

```
*
```

```
* This allows dates to be compared numerically while preserving
```

```
* chronological ordering.
```

```
*/
```

```
int calculateDateValue()
```

```
{
```

```
    int month = stoi(date.substr(0, 2));    // Extract the month.
```

```
    int day = stoi(date.substr(3, 2));      // Extract the day.
```

```
    int year = stoi(date.substr(6, 4));     // Extract the year.
```

```
    return year * 10000 + month * 100 + day; // Return YYYYMMDD.
```

```
}
```

```
/*
```

```
* Returns true when this email should appear before "other"
```

```
* in the MaxHeap.
```

```
*/
```

```
bool hasHigherPriority(const Email& other) const
```

```
{
```

```
    // Sender category has the highest importance.
```

```

    if (priority != other.priority)
    {
        return priority > other.priority;
    }

    // If sender categories match, the newest email comes first.
    if (dateValue != other.dateValue)
    {
        return dateValue > other.dateValue;
    }

    // If everything else is equal, use insertion order to keep
    // the behavior deterministic.
    return sequence > other.sequence;
}

/*
 * Displays the email in the exact format required by the assignment.
 */

void display() const
{
    cout << "Sender: " << sender << endl;
    cout << "Subject: " << subject << endl;
    cout << "Date: " << date << endl;
}

```

```
};
```

```
/*
```

```
 * MaxHeap
```

```
 *
```

```
 * Implements a MaxHeap using a vector as the underlying list.
```

```
 *
```

```
 * The largest/highest-priority Email is always stored at index 0.
```

```
 *
```

```
 * Parent index:
```

```
 *  $(\text{index} - 1) / 2$ 
```

```
 *
```

```
 * Left child:
```

```
 *  $2 * \text{index} + 1$ 
```

```
 *
```

```
 * Right child:
```

```
 *  $2 * \text{index} + 2$ 
```

```
 */
```

```
class MaxHeap
```

```
{
```

```
private:
```

```
    vector<Email> heap; // List-based storage for the MaxHeap.
```

```
/*
```

* Moves an element upward until the MaxHeap property is restored.

*

* This is used after inserting a new email.

*/

```
void heapifyUp(int index)
```

```
{
```

```
    // Continue while the current element has a parent.
```

```
    while (index > 0)
```

```
    {
```

```
        int parent = (index - 1) / 2; // Calculate parent index.
```

```
        // Stop if the parent already has higher priority.
```

```
        if (!heap[index].hasHigherPriority(heap[parent]))
```

```
        {
```

```
            break;
```

```
        }
```

```
        // Swap the child with its parent.
```

```
        swap(heap[index], heap[parent]);
```

```
        // Continue checking from the parent's old position.
```

```
        index = parent;
```

```
    }
```

```
}
```

```

/*
 * Moves an element downward until the MaxHeap property is restored.
 *
 * This is used after removing the highest-priority email.
 */
void heapifyDown(int index)
{
    int size = static_cast<int>(heap.size());

    while (true)
    {
        int left = 2 * index + 1;    // Calculate left child index.
        int right = 2 * index + 2;    // Calculate right child index.
        int largest = index;          // Assume current node is largest.

        // Check whether the left child has higher priority.
        if (left < size &&
            heap[left].hasHigherPriority(heap[largest]))
        {
            largest = left;
        }

        // Check whether the right child has higher priority.
        if (right < size &&

```

```

        heap[right].hasHigherPriority(heap[largest]))
    {
        largest = right;
    }

    // The heap property is already correct.
    if (largest == index)
    {
        break;
    }

    // Move the higher-priority child upward.
    swap(heap[index], heap[largest]);

    // Continue from the child's old position.
    index = largest;
}
}

```

public:

/*

* Adds an email to the MaxHeap.

*

* Time complexity: $O(\log n)$

*/

void insert(const Email& email)

{

 // Place the new email at the end of the list.

 heap.push_back(email);

 // Restore the MaxHeap property.

 heapifyUp(static_cast<int>(heap.size()) - 1);

}

/*

* Returns whether the heap contains no emails.

*

* Time complexity: $O(1)$

*/

bool empty() const

{

 return heap.empty();

}

/*

* Returns the number of unread emails.

*

* Time complexity: $O(1)$

*/

int size() const

{

 return static_cast<int>(heap.size());

}

/*

* Returns the highest-priority email without removing it.

*

* This is what NEXT needs because two NEXT commands in a row

* must display the same email.

*

* Time complexity: $O(1)$

*/

const Email& top() const

{

 return heap[0];

}

/*

* Removes the highest-priority email from the heap.

*

* Time complexity: $O(\log n)$

*/

void removeTop()

{

 // Do nothing if there are no emails.

 if (heap.empty())

 {

 return;

 }

 // If only one email exists, simply remove it.

 if (heap.size() == 1)

 {

 heap.pop_back();

 return;

 }

 // Move the final element into the root position.

 heap[0] = heap.back();

 // Remove the duplicate final element.

 heap.pop_back();

 // Restore the MaxHeap property.

```

        heapifyDown(0);
    }
};

/*
 * EmailManager
 *
 * Controls the interaction between the input commands and the MaxHeap.
 * Keeping this logic separate from the heap makes the program easier
 * to maintain and test.
 */
class EmailManager
{
private:
    MaxHeap emailQueue;    // Stores all unread emails.
    long long nextSequence; // Tracks insertion order for tie-breaking.

    /*
     * Creates an Email object from an EMAIL command.
     */
    Email createEmail(const string& line)
    {
        // Remove the "EMAIL " portion from the beginning of the line.
        string emailData = line.substr(6);
    }

```

```
// Find the commas separating the three fields.

size_t firstComma = emailData.find(',');

size_t secondComma = emailData.find(',', firstComma + 1);


// Extract the sender category.

string sender = emailData.substr(0, firstComma);


// Extract the subject between the two commas.

string subject = emailData.substr(
    firstComma + 1,
    secondComma - firstComma - 1
);


// Extract the date after the second comma.

string date = emailData.substr(secondComma + 1);


// Create the Email object with its insertion sequence.

Email email(sender, subject, date, nextSequence);


// Increment the sequence for the next email.

nextSequence++;


return email;
}
```

public:

/*

* Constructor:

* Starts the insertion sequence at zero.

*/

EmailManager()

{

 nextSequence = 0;

}

/*

* Processes one input command.

*/

void processCommand(const string& line)

{

 // EMAIL commands add a new email to the priority queue.

 if (line.rfind("EMAIL ", 0) == 0)

 {

 Email email = createEmail(line);

 emailQueue.insert(email);

 }

```
// NEXT displays the highest-priority email without removing it.
```

```
else if (line == "NEXT")
```

```
{
```

```
    if (!emailQueue.empty())
```

```
    {
```

```
        cout << "Next email:" << endl;
```

```
        emailQueue.top().display();
```

```
    }
```

```
}
```

```
// READ removes the highest-priority email without displaying it.
```

```
else if (line == "READ")
```

```
{
```

```
    emailQueue.removeTop();
```

```
}
```

```
// COUNT displays the current number of unread emails.
```

```
else if (line == "COUNT")
```

```
{
```

```
    cout << "There are "
```

```
        << emailQueue.size()
```

```
        << " emails to read."
```

```
        << endl;
```

```
}
```

```

    }
};

/*
 * Main program
 *
 * Reads commands until the input file reaches its end.
 */
int main()
{
    EmailManager manager; // Create the object responsible for email management.
    string line;           // Stores one complete input line at a time.

    // Read and process every command from standard input.
    while (getline(cin, line))
    {
        manager.processCommand(line);
    }

    return 0; // Indicate successful program completion.
}

```

Claude:

```

/*
* Program:    EECS 348 Assignment 3 - CEO Email Prioritizer
* Description: C++ program that prioritizes emails for a busy CEO using a
*              hand-written, array-based MaxHeap acting as a priority queue.
*              Priority: Boss > Subordinate > Peer > ImportantPerson > OtherPerson;
*              within the same category, the NEWEST email comes first.
* Inputs:     Test file path (argv[1], or stdin if omitted). Commands:
*              EMAIL <category>,<subject>,<MM-DD-YYYY>
*              NEXT | READ | COUNT
* Output:     Terminal output for NEXT and COUNT (see printEmail / main).
* Collaborators: None
* Other sources: Code generated by Claude (Anthropic LLM), then reviewed by author.
* Author:     <Your Full Name>
* Creation date: <MM-DD-YYYY>
* Revision date: <MM-DD-YYYY>
* Revisions:   Initial version.
*
* Complexity:  EMAIL O(log n), NEXT O(1), READ O(log n), COUNT O(1).
*              Space O(n) (one heap slot per unread email).
*
* Authorship key used in block comments below:
*  [AI] = generated by Claude, [ME] = author-written/modified.
*/

#include <stdint> // fixed-width integer types
#include <stdio>  // sscanf

```

```

#include <fstream> // reading the test file

#include <iostream> // terminal I/O

#include <string> // std::string

#include <utility> // std::move, std::swap

#include <vector> // contiguous storage for the heap array

// -----

// [A] Email: one message. Priority is packed into a single integer so every
// heap comparison is O(1) integer math instead of string/date comparison.
// -----

class Email {
public:
    Email(std::string category, std::string subject, std::string date,
          int rank, int dateKey, std::uint64_t seq)
        : category_(std::move(category)), subject_(std::move(subject)),
          date_(std::move(date)),
          key_(static_cast<std::int64_t>(rank) * kRankScale + dateKey),
          seq_(seq) {}

    // Strict "higher priority than" test used by the heap.
    // Larger key wins (better category, then newer date); ties go to the
    // email that arrived first (smaller seq_) so behavior is deterministic.
    bool higherPriorityThan(const Email& o) const {
        return key_ != o.key_ ? key_ > o.key_ : seq_ < o.seq_;
    }
}

```

```

// Read-only accessors for display.

const std::string& category() const { return category_; }

const std::string& subject() const { return subject_; }

const std::string& date() const { return date_; }

private:

    static constexpr std::int64_t kRankScale = 100000000LL; // > any YYYYMMDD value

    std::string category_; // sender category as given in the file

    std::string subject_; // subject line (may contain spaces)

    std::string date_; // original MM-DD-YYYY text, echoed on output

    std::int64_t key_; // rank * 1e8 + YYYYMMDD

    std::uint64_t seq_; // arrival order, used only to break exact ties

};

// -----

// [AI] MaxHeap: array-based binary heap, built from scratch (no std::*heap).

// Element at index i has children 2i+1 and 2i+2 and parent (i-1)/2.

// T must provide higherPriorityThan(const T&).

// -----

template <typename T>

class MaxHeap {

public:

    bool empty() const { return data_.empty(); }

    std::size_t size() const { return data_.size(); }

    // O(log n): append at the end, then bubble up.

```

```
void push(T item) {  
    data_.push_back(std::move(item));  
    siftUp(data_.size() - 1);  
}
```

// O(1): highest-priority element. Caller must ensure !empty().

```
const T& top() const { return data_.front(); }
```

// O(log n): remove root by moving the last element there, then sift down.

// Caller must ensure !empty().

```
void pop() {  
    data_.front() = std::move(data_.back()); // last element replaces root  
    data_.pop_back();           // shrink the array  
    if (!data_.empty()) siftDown(0); // restore heap order  
}
```

private:

```
std::vector<T> data_; // the heap array
```

// Move element at i up while it outranks its parent.

```
void siftUp(std::size_t i) {  
    while (i > 0) {  
        std::size_t parent = (i - 1) / 2;  
        if (!data_[i].higherPriorityThan(data_[parent])) break;  
        std::swap(data_[i], data_[parent]);  
        i = parent;  
    }
```

```

    }
}

// Move element at i down while a child outranks it.
void siftDown(std::size_t i) {
    const std::size_t n = data_.size();
    for (;;) {
        std::size_t best = i, l = 2 * i + 1, r = l + 1;
        if (l < n && data_[l].higherPriorityThan(data_[best])) best = l;
        if (r < n && data_[r].higherPriorityThan(data_[best])) best = r;
        if (best == i) break;      // heap property holds
        std::swap(data_[i], data_[best]);
        i = best;
    }
}

};

// -----
// [AI] Ceolnbox: owns the heap and implements the four commands.
// -----

class Ceolnbox {
public:
    // Handle "EMAIL <category>,<subject>,<date>"; args is everything after "EMAIL ".
    // Returns false (and stores nothing) if the line is malformed.
    bool addEmail(const std::string& args) {
        std::size_t c1 = args.find(',');      // end of category

```

```

std::size_t c2 = args.rfind(';');           // start of date

if (c1 == std::string::npos || c2 == c1) return false; // need two commas

std::string cat = args.substr(0, c1);

std::string subj = args.substr(c1 + 1, c2 - c1 - 1);

std::string date = args.substr(c2 + 1);


int rank = rankOf(cat);

int m, d, y;

if (rank < 0 || std::sscanf(date.c_str(), "%d-%d-%d", &m, &d, &y) != 3)

    return false;           // bad category/date


heap_.push(Email(cat, subj, date, rank, y * 10000 + m * 100 + d, nextSeq_++));

return true;

}


// NEXT: show (but keep) the highest-priority email.

void next() const {

    if (heap_.empty()) { std::cout << "No emails to read.\n"; return; }

    const Email& e = heap_.top();

    std::cout << "Next email:\n"

        << "Sender: " << e.category() << '\n'

        << "Subject: " << e.subject() << '\n'

        << "Date: " << e.date() << '\n';

}


// READ: discard the highest-priority email; silently ignored if empty.

```

```

void read() { if (!heap_.empty()) heap_.pop(); }

// COUNT: number of unread emails.

void count() const {
    std::cout << "There are " << heap_.size() << " emails to read.\n";
}

private:
    MaxHeap<Email> heap_;    // the priority queue
    std::uint64_t nextSeq_ = 0; // arrival counter for tie-breaking

    // Map category name to rank (higher = read sooner); -1 if unknown.
    static int rankOf(const std::string& c) {
        if (c == "Boss")      return 4;
        if (c == "Subordinate") return 3;
        if (c == "Peer")      return 2;
        if (c == "ImportantPerson") return 1;
        if (c == "OtherPerson") return 0;
        return -1;
    }
};

// -----
// [A1] main: stream the input line by line and dispatch commands.
// -----

int main(int argc, char* argv[]) {

```

```

std::ifstream file;           // used only if a path is given
if (argc > 1) {
    file.open(argv[1]);
    if (!file) { std::cerr << "Cannot open " << argv[1] << '\n'; return 1; }
}

std::istream& in = (argc > 1) ? static_cast<std::istream&>(file) : std::cin;

CeolInbox inbox;

std::string line;

const std::string kEmail = "EMAIL ";    // command prefix (note trailing space)

while (std::getline(in, line)) {
    if (!line.empty() && line.back() == '\r') line.pop_back(); // tolerate CRLF files

    if (line.compare(0, kEmail.size(), kEmail) == 0) {
        if (!inbox.addEmail(line.substr(kEmail.size())))
            std::cerr << "Skipping malformed line: " << line << '\n';
    } else if (line == "NEXT") inbox.next();
    else if (line == "READ") inbox.read();
    else if (line == "COUNT") inbox.count();

    // Blank/unrecognized lines are ignored.
}

return 0;
}

```

- e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude: Fully correct as far as I tested.

ChatGPT: Fully correct as far as I tested.

Verdict: I’m sure I missed an edge case somewhere, but I got no crashes anywhere.

Both are good enough, but Claude’s input is much more flexible than ChatGPT’s.

Execution Time:

Claude:

EMAIL: $O(\log n)$

NEXT: $O(1)$

READ: $O(\log n)$

COUNT: $O(1)$

ChatGPT:

EMAIL: $O(\log n)$

NEXT: $O(1)$

READ: $O(\log n)$

COUNT: $O(1)$

Verdict: Claude’s may be very slightly faster due to one simple comparison in the “hasHigherPriority” and the “higherPriorityThan” functions.

Space Complexity:

Claude: Space complexity is $O(n)$. Stores five variables.

ChatGPT: Space complexity is $O(n)$. Stores six variables.

Verdict: Claude’s is barely better.

Maintainability:

Claude: Commented well enough

ChatGPT: Commented very well

Verdict: ChatGPT's comments are better than Claude's

- f. Select one to use as the basis for this assignment and justify why you picked it

I chose Claude's code because it has slightly better space complexity and slightly better time complexity. I also really like how flexible the input is for Claude's code.

- g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I added a Makefile for testing.

Execution Time: Nothing

Space Complexity: Nothing

Maintainability: I split the classes into separate .h and .CPP files for easier readability.